

Reproducing Multi-Agent System Failures: ChatDev 1.0 and Magentic-One under gpt-5.4-mini (2-page summary)

MAS memory research — working report

June 10, 2026

1 Dataset construction

The 30-task dataset is deliberately biased toward inter-agent misalignment and is not a representative benchmark sample. The public per-trace MAST annotations (HF mcmri/MAD) are a misjoin (206 unique rows for 1,242 traces), so we recovered ChatDev human annotations from a deleted file in the MAST repo’s git history and computed Magentic-One outcomes against GAIA gold answers (120/165 original runs failed). Each candidate’s *original GPT-4o trace* was then close-read and kept only if its failure had a category-2 (inter-agent) surface. Final composition: 15 ChatDev game tasks (4 high / 4 medium / 4 low cat-2 prior, 3 solved controls) and 15 Magentic-One GAIA tasks (1/9/3 + 2 controls).

2 The two architectures

ChatDev 1.0 is a waterfall “software company”: a fixed phase chain (DemandAnalysis → Coding → CodeReview×3 → Test → docs), each phase a two-agent role-played dialogue. Inter-phase memory is a *shared code store* filled by a regex extractor over chat messages. Two load-bearing flaws: completion budgets are 4096 – prompt tokens (long emissions truncate mid-file, the truncation signal is never checked), and the extractor silently drops unterminated code blocks. **Magentic-One** is a star topology: an orchestrator keeps task/progress ledgers and delegates single steps to four specialists (multimodal WebSurfer, FileSurfer, Coder, ComputerTerminal), rewriting the ledger from its own summary when progress stalls.

Reproduction setup. Both systems run their official, unmodified code (ChatDev v1.1.6; Magentic-One via the MAST authors’ agbench scenarios with AutoGen 0.4.8). The only swap is the model: a local proxy translates their chat-completions calls to Perplexity’s Responses API serving gpt-5.4-mini (tool calls, JSON mode, and screenshot vision verified passing through). Deviations: host-local execution instead of Docker; GAIA web drift since 2024.

3 Outcomes vs. the original GPT-4o runs

Magentic-One: 2/15 → 3/15 solved (one flip to pass, no regressions, both controls reproduce). ChatDev: 3/15 → 5/15 (two flips; all three controls reproduce; caveat: original grades are human, reproduction grades are our judge). **The selected failure set largely survives a two-generation model upgrade**, consistent with structural rather than capability-bound causes.

4 MAST taxonomy statistics

A two-tier gpt-5.5 judge (temp. 0) labels every trace: tier 1 reads with no taxonomy and emits open-ended findings; tier 2 emits per-mode binary flags, each requiring a verbatim quote. Calibration on four ground-truth anchors: commission modes (2.3–2.6, 3.x) reliable — including 2.4/2.5, which both MAST’s o1 judge and our earlier gpt-5.4 judge emitted *zero* times — omission modes (2.1/2.2) ≈50% recall, so their counts are lower bounds.

Mode	Name	ChatDev (n=15)	Magentic-One (n=15)
2.1	Conversation Reset	7	4
2.2	Fail to Ask for Clarification	5	8
2.3	Task Derailment	6	8
2.4	Information Withholding	15	4
2.5	Ignored Other Agent’s Input	15	13
2.6	Action–Reasoning Mismatch	12	13
3.1–3.3	Verification failures	13/13/13	13/13/13

The category-2 profiles are opposite and match the topologies: ChatDev flags 2.4 in *every* run — including the three that produced working software — because the code-persistence channel loses artifacts in every run; success depends only on *which* artifacts are lost. Magentic-One is 2.5-heavy but 2.4-light: the hub sees every message (little to withhold) but routinely discards or overrides specialist input. Rubber-stamp verification is near-universal in both.

5 Open-ended judge trends

Tier-1 produced ~650 free-form findings; after excluding reproduction-harness artifacts, three mechanism chains dominate. (1) *ChatDev’s lossy persistence chain*: budget-truncated code emission → extractor silently drops the file → store goes stale → reviewer repeats an identical comment three cycles → tester rubber-stamps → manual documents features that don’t exist (clusters: truncated output 17, state corruption 10, scope drift 11, lost artifact 5). (2) *ChatDev error masking*: when a missing artifact finally breaks the build, the programmer patches *around* it (try/except, installing an unrelated PyPI package matching the module name) instead of re-emitting it. (3) *Magentic-One unsupported finals*: the orchestrator answers from prior knowledge or partial evidence (7 traces), skips planned delegation (unused tools, 5), and drops working state at ledger re-plans.

6 Four case briefs (selected for inter-agent misalignment relevance)

ChatDev: DouDizhuPoker (failed; modes 2.4, 2.5, 2.6, 3.x)

Task: build a playable Dou Dizhu card game with a GUI. The Programmer emitted `gui.py` repeatedly, but every emission hit the token budget mid-statement and the extractor silently dropped it — only `main.py` and `game_logic.py` ever reached the shared store (2.4). The reviewer correctly demanded `gui.py` for three consecutive cycles; the Programmer kept “complying” while producing the same two files (2.5), claiming a complete GUI implementation each time (2.6). Testing hit the inevitable `ModuleNotFoundError` and masked it with a CLI fallback; a launch-only test then passed (3.3). The full chain — truncation → silent drop → ignored review → masked error → rubber stamp — in one trace.

ChatDev: Gomoku (control, *succeeded*; modes 2.4, 2.5, 2.6)

A solved control that still exhibits the channel failure: the extractor mis-captured `board.py` as the literal shell command `python main.py`, losing a correct Board implementation (2.4). Review caught it and the file was regenerated, but later cycles ignored a “no changes needed” signal (2.5) and a reviewer alleged a coordinate bug while simultaneously stating the coordinates were consistent (2.6). Misalignment fired; the task survived only because the lost artifact happened to be recoverable — evidence that 2.4 is a property of the channel, not of task failure.

Magentic-One 5a0c1adf (GAIA, failed; modes 2.5, 2.6, 3.x)

Task: find which post-1977 Malko Competition winner’s recorded nationality is a now-defunct country. The orchestrator’s plan was sound and it issued correct, specific navigation instructions (open the winners archive on `malkocompetition.dk`); WebSurfer repeatedly ignored them, re-running dead-end searches and never opening the archive (2.5). After the loop stalled, the orchestrator violated its own stated “do not guess before exhaustive filtering” rule and returned an unsupported answer (2.6, 3.1). Hub-spoke signature: information flowed, but the spoke didn’t execute and the hub didn’t verify.

Magentic-One 72e110e7 (GAIA, failed; modes 2.1, 2.4, 2.5, 2.6)

Task: a BASE (DDC 633) bibliographic lookup. The target page rendered blank; WebSurfer only “waited” despite explicit instructions to inspect source or pivot (2.5). The orchestrator then entered a re-planning loop, repeatedly rewriting its fact sheet and restarting the task framing instead of executing (2.1) — and although its rewritten plans correctly named the fix (archived copies, source inspection), it never delegated those steps to any agent (2.4: the hub withheld its own recovery plan from execution) before answering “unknown” (2.6). The ledger-reset mechanism converting a recoverable tool failure into task failure.

Full 34-page version with per-task outcome table, all 14 modes, trend-cluster table, and 30 case briefs: `main.pdf` in the same directory. M. Cemri et al., “Why Do Multi-Agent LLM Systems Fail?” arXiv:2503.13657 (2025).